



15. K-Means Clustering

Grouping Data Based on Similarities

Previous Section:

[14. K-Nearest Neighbors](#)

Contents:

[1. K-Means Clustering — The Workflow](#)

[2. K-Means Clustering with Python](#)

[3. K-Means Clustering - The Elbow Method](#)

[4. K-Means Clustering - The Silhouette Method](#)

[Summary](#)

[References](#)

Now we're stepping into something a little different from what has been covered before. This section is about **K-Means Clustering**.

And unlike many of the models we've explored so far—Decision Tree, KNN, Random Forest, Logistic Regression—this one is **unsupervised**. Which means, there is nothing to predict. No labels. No "correct answers". No teacher telling the model what is right or wrong. Instead, the model is given data, and simply asked: ***"What patterns can you find?"***

Let's make this concrete. Suppose we have a dataset of university students, and for each student, we only know their **birthplace**. That's it. No labels like "North", "Central", or "South". Just raw data.

Now we apply K-Means with $K = 3$. What happens? The algorithm will divide the students into **three groups** based on similarity. And interestingly, those groups might naturally correspond to: Northern region; Central region; and Southern region.

But here's the key idea—and I really want you to notice this: It may *look* like classification, but it's not. Because:

- We never told the model what "North" or "South" means
- We never gave it labels
- We never trained it to predict anything

It simply **used all available data** and **grouped them based on patterns**.

Now here's where it gets even more important. In supervised learning:

- You train on existing labeled data
- Then you use the model to predict **new data**

But in an unsupervised learning model like K-Means:

- **Every single data point is used in the grouping process**
- There is no concept of "training vs testing" in the same way
- If a new data point appears later, it doesn't automatically belong anywhere
→ You would need to re-run or assign it based on the clusters

And that is exactly the essence of an **unsupervised algorithm**. No labels. No predictions. Just structure, discovered from data itself.

1. K-Means Clustering — The Workflow

(This is where everything comes together. I don't usually go straight to the workflow at the first section. K-Means is just that special!!)

Now that we understand what K-Means is trying to do, let's walk through **how it actually works**.

This is the most important part of this section—so don't rush it. K-Means may look simple on the surface, but every step carries a very specific purpose.

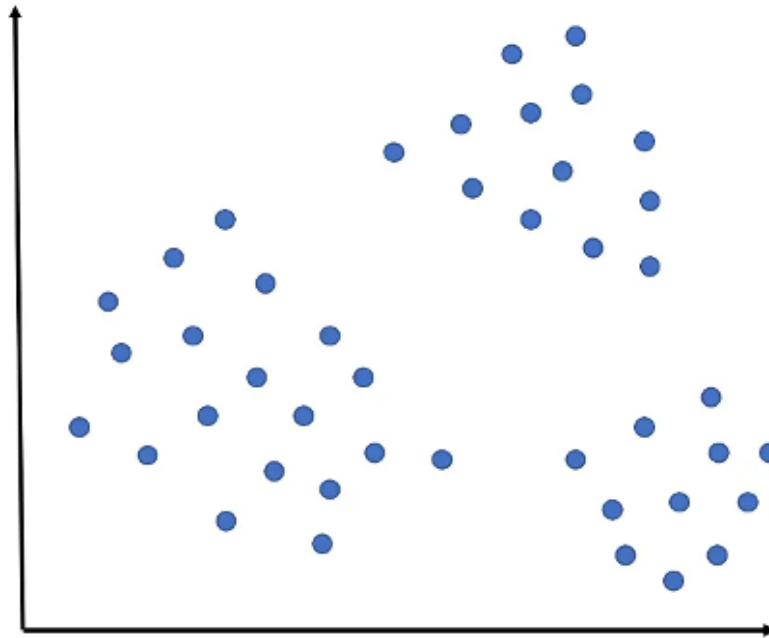
Before we begin, we need to reinstate the core idea of this algorithm: K-Means is a clustering algorithm that partitions data into **K groups**, where each group is represented by a **centroid**. Think of a centroid as the *center of gravity* of a cluster.

And the objective is very clear:

Minimize the distance between each data point and the centroid of the cluster it belongs to.

In other words, points should stay as close as possible to “their center.”

Now imagine we are given a 2D space—just points scattered across a plane.



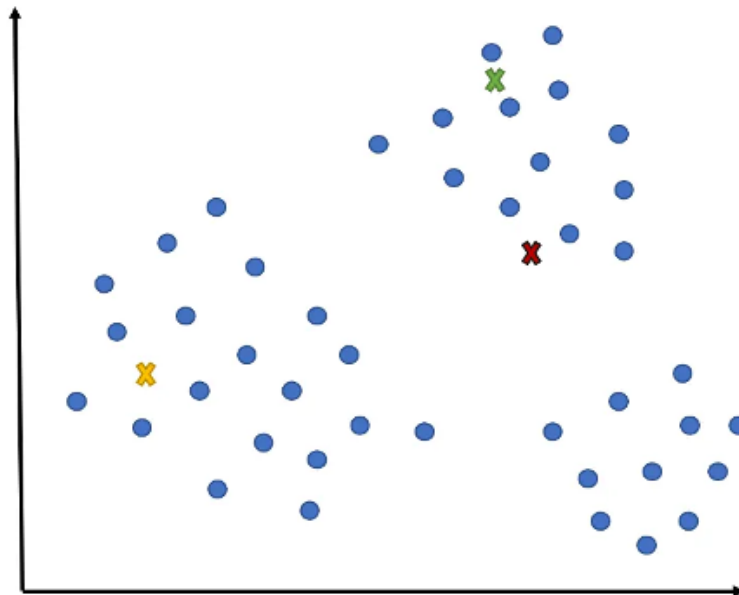
No labels. No colors. Just coordinates. We’re going to let K-Means **discover structure from chaos**.

- **Step 1 — Define the Number of Clusters (K):** Before anything begins, we must make a decision.
 - **How many clusters do we want?** This number is denoted as K . In this example: We choose $K = 3$
 - Which means we want to divide the data into **3 groups** → The algorithm will create **3 centroids** → The space will be partitioned into **3 regions**



Important note: K-Means does **not** figure out K for you. You decide it beforehand. So already, there is a bit of human judgment involved.

- **Step 2 — Randomly Initialize the Centroids:** Now comes the part that feels a bit chaotic. We randomly place K **centroids** into the space.
 - Since $K = 3$, we pick **3 initial centroids**.
 - These centroids are often randomly positioned. They do NOT have to be actual data points, and simply act as initial “guesses” of where cluster centers might be.



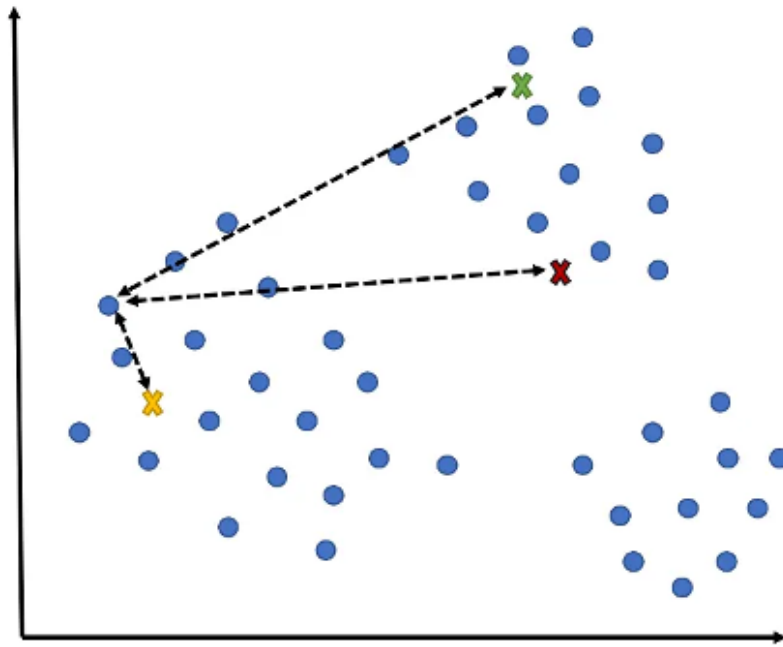
At this moment, nothing is structured yet. Just scattered data points and randomly placed centroids.

- **Step 3 — Assign Each Point to the Nearest Centroid:** Now the algorithm begins to **impose structure**.

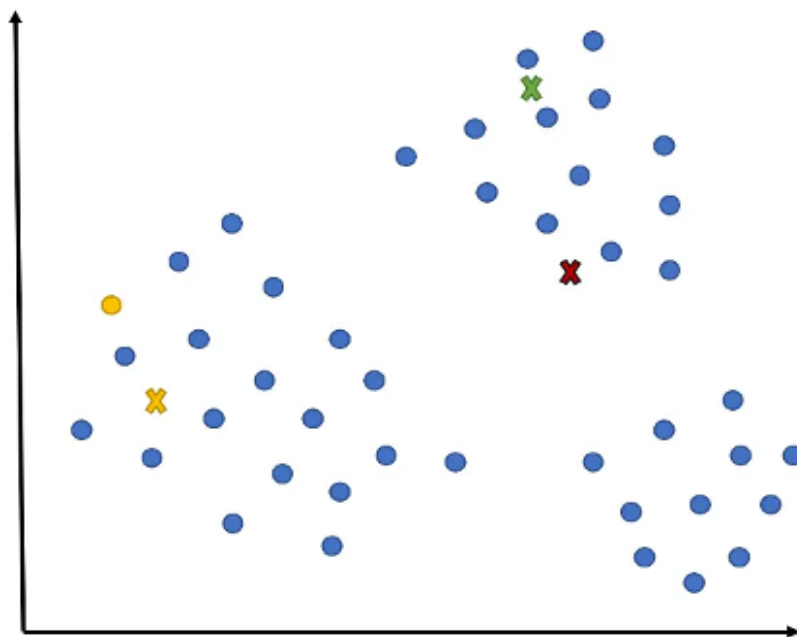
Every data point will decide: *“Which centroid am I closest to?”*

- **Step 3a — Compute Distance:** Since we are in 2D space, each point has coordinates (x, y) .
 - To measure closeness, we use **Euclidean distance**.
 - For each data point: we compute its distance to **every centroid**, and compare those distances

$$d(p, c) = \sqrt{(x_p - x_c)^2 + (y_p - y_c)^2}$$



- **Step 3b — Choose the Closest Centroid (argmin):**



Now comes a very important concept: *argmin*.

- Instead of asking: "What is the smallest distance?". We ask: **"Which centroid gives the smallest distance?"**
- Formally using the formula below. This returns the **index of the closest centroid**.

$$\operatorname{argmin}_{i \in \{1, \dots, K\}} d(p, c_i)$$



Let me slow this down with intuition: *min* → gives the value, but *argmin* → gives *where that value happens*.

Example: *Distances* : 5, 12, 8 → *min* = 5 → *argmin* = 1 (the first centroid)

So each point:

- Finds the nearest centroid
- Gets assigned to that cluster
- Takes on its "color" (visually speaking)

This step is repeated for every point. Totally, every point belongs to one cluster
→ Structure has started to emerge.

- **Step 4 — Recompute the Centroids (Find the True Centers):** Now here's the twist. Those initial centroids? They were just guesses. Now that clusters exist, we can compute their **actual centers**.

- **Step 4a — Compute Mean of X**

$$\mu_x = \frac{\sum x_i}{N}$$

- **Step 4b — Compute Mean of Y**

$$\mu_y = \frac{\sum y_i}{N}$$

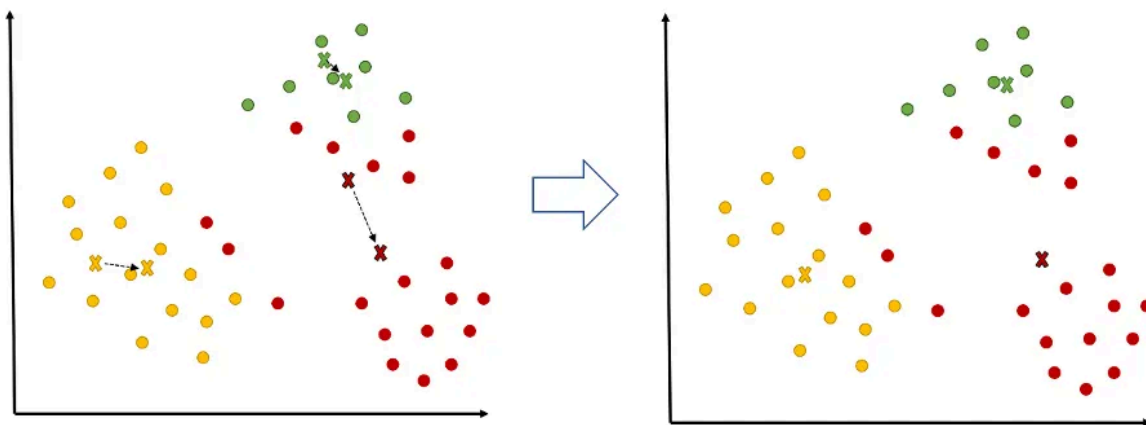
- **Step 4c — Form the New Centroid**

$$\mu = (\mu_x, \mu_y)$$

So each cluster gets a **new centroid**, which is the average position of all its points. And now something important happens: The centroids **move**. They shift from random guesses to actual centers.

- **Step 5 — Reassign Points (Again)**

Now that centroids have moved. We repeat the assignment process. Each point once again asks: ***"Which centroid am I closest to now"*** → And clusters get updated.



A helpful way to visualize this (as what I have been doing): Imagine all points lose their colors. Then get recolored based on new centroids. Some points will stay in the same cluster. Others will switch.

- **Step 6 — Repeat Until Convergence**

Now we loop: Step 4 → Update centroids and Step 5 → Reassign points → Again, and again, and again...

Until **the centroids stop moving**. Or move only by a very tiny amount.

This is called: **Convergence**

At convergence. The clusters stabilize. Assignments no longer change → The structure is finalized

- **Final Result — Order from Chaos**

What started as random centroids and unlabeled data becomes clearly defined clusters, well-positioned centroids, and organized structure.

K-Means is not “intelligent” in the way supervised models are. It doesn’t learn from truth. It doesn’t correct mistakes. Instead, it **refines structure through iteration**.

Each loop is not learning from error. But from **rebalancing geometry**.

And that’s the beauty of it: No labels. No supervision. Just distance, and the relentless pursuit of better centers.

Now that you’ve seen the full workflow, there are a few questions that almost always come up. And they’re not just “extra details”—they actually define how well K-Means works in practice.

So let’s go through them carefully.

- **Question 1 — How do we choose the number of centroids (K)?** This is the uncomfortable part. Because K-Means does **not** tell you what K should be. You must decide it *before* the algorithm even begins. Let’s think about what K really controls:
 - **Small K** → Few clusters → Large, general groups → You risk **underfitting** (too simple, missing structure)
 - **Large K** → Many clusters → Small, fragmented groups → You risk **overfitting** (too specific, capturing noise)

So the goal is not just to pick K . It’s to find a balance between simplicity and detail.

- **Question 2 — Why use the mean instead of existing data points as centroids?:** This is a very important design choice in K-Means. After forming clusters, we compute the **mean position**. And use that as the new centroid. Why? Because the mean represents the **true center of mass**. It is not biased toward any single data point.
 - Imagine a cluster shaped like a cloud. If you pick an existing point → it might sit near the edge. If you take the mean → it lands right in the middle.
 - This gives us two major advantages:
 - **Better Representation:** The centroid reflects the entire cluster, not just one example.

- **Smoother Movement:** Centroids can move **freely in space**, not just jump between points. Which leads to faster convergence, more stable clusters, and better overall quality
- So the mean is not just a mathematical choice. It is what allows K-Means to continuously refine itself.
- **Question 3 — Can centroids be forced to be actual data points?** You could. But the moment you do that. You are no longer using K-Means. You are using a different algorithm: **K-Medoids**.
 - In K-Medoids: Centroids must be **existing data points**. Each cluster is represented by a real example (called a *medoid*)
 - This has one advantage: It can be more **robust to noise and outliers**. Because extreme values don't pull the center away like the mean does.
 - But there is a trade-off. A very real one: **Computation becomes much more expensive**. Why? Because instead of computing a simple average. The algorithm must compare distances between many pairs of points and evaluate which actual point best represents the cluster
 - So for: Small datasets → K-Medoids can work well; Large datasets (like images) → It becomes impractical

These three questions reveal something deeper. K-Means is not just an algorithm. It is a **set of assumptions**:

- You assume a fixed number of clusters
- You assume clusters are centered around a mean
- You assume distance defines similarity

And when those assumptions hold, K-Means feels almost effortless. But when they don't, you start needing different tools. So don't just memorize how K-Means works. Understand what it *believes* about the world. Because every model is just a perspective in disguise.

2. K-Means Clustering with Python

Before we jump into the code, let's quickly understand what we're working with.

We are using the following housing dataset:

housing.csv

The dataset has three features:

- `longitude` → how far west a house is
- `latitude` → how far north a house is
- `median_house_value` → the median price of houses in that area

The Goal: We want to group houses into clusters such that houses in the same cluster are similar in **location value**

Since K-Means is an **unsupervised algorithm**: There is **no target variable**. No “correct answers”. No train/test split → We are not predicting anything. Instead, we are letting the model **discover natural groupings in the data based on similarity**.

That’s all we need to know. Now let’s see how this idea turns into code:

```
# K-Mean Clustering for Housing
# Unsupervised Learning: No data split needed
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import seaborn as sns
import matplotlib.pyplot as plt

# Read data
df = pd.read_csv('housing.csv', usecols=['longitude', 'latitude', 'median_house_value'])

# Cluster by geography
X = df[['longitude', 'latitude']]
scaler = StandardScaler() # Normalize the data
X_scaled = scaler.fit_transform(X)
# Fitting the model
model = KMeans(n_clusters=3, random_state=0, n_init='auto')
model.fit(X_scaled)

# New column that stores the cluster
```

```

df['cluster'] = model.labels_

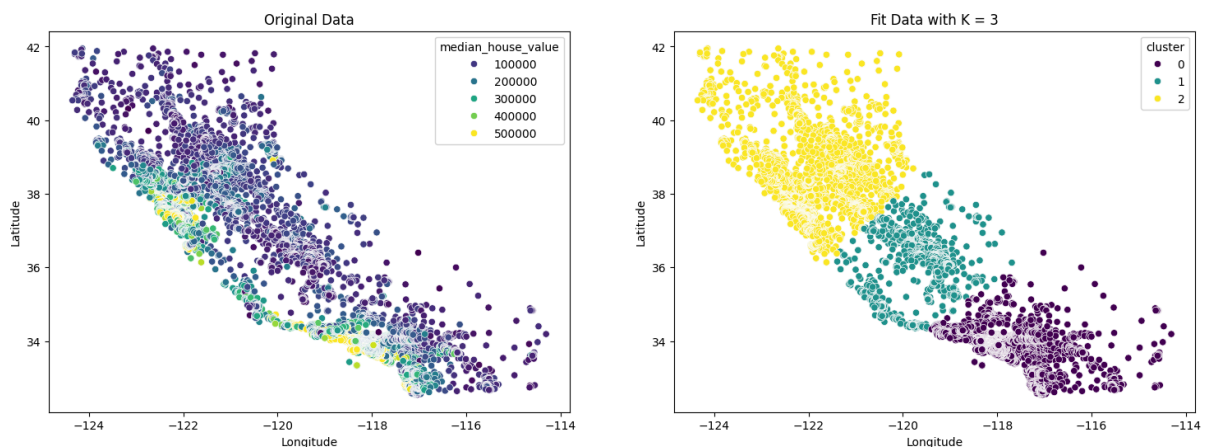
# Visualize the original data and fit data
fig, axes = plt.subplots(1, 2, figsize=(18, 6))
sns.scatterplot(data=df, x='longitude', y='latitude', hue
='median_house_value',
                palette='viridis', ax=axes[0])

axes[0].set_title('Original Data')
axes[0].set_xlabel('Longitude')
axes[0].set_ylabel('Latitude')

sns.scatterplot(data=df, x='longitude', y='latitude', hue
='cluster',
                palette='viridis', ax=axes[1])
axes[1].set_title('Fit Data with K = 3')
axes[1].set_xlabel('Longitude')
axes[1].set_ylabel('Latitude')

plt.show()

```



You can increase the number of K in the `n_clusters` parameter if 3 isn't to your liking:

```

# Fitting the model
model2 = KMeans(n_clusters=5, random_state=0, n_init='auto')

```

```

model2.fit(X_scaled)

# New column that stores the cluster
df['cluster_2'] = model2.labels_

model3 = KMeans(n_clusters=7, random_state=0, n_init='auto')
model3.fit(X_scaled)
df['cluster_3'] = model3.labels_

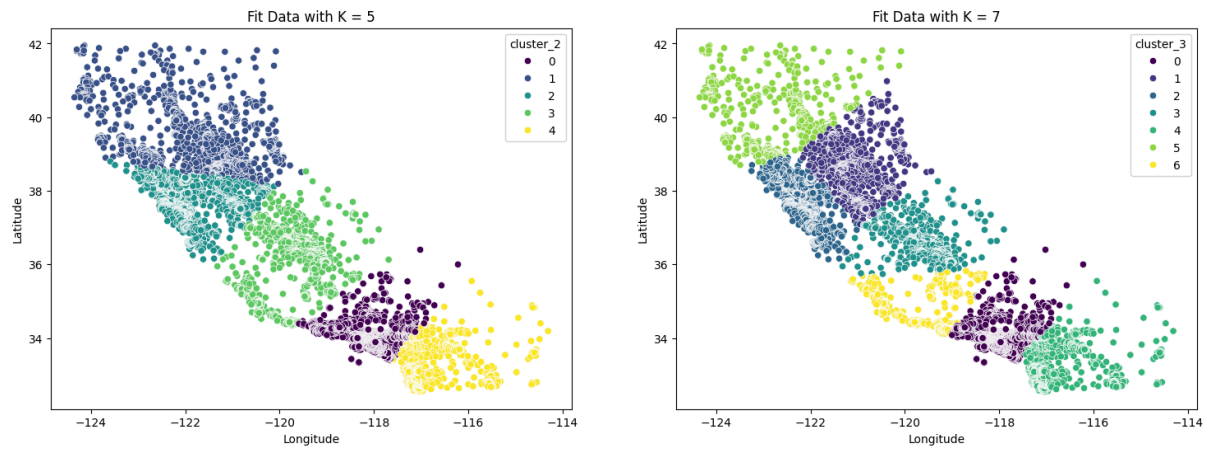
# Plot
fig, axs = plt.subplots(1, 2, figsize=(18, 6))

sns.scatterplot(data=df, x='longitude', y='latitude', hue='cluster_2',
                palette='viridis', ax=axs[0])
axs[0].set_title('Fit Data with K = 5')
axs[0].set_xlabel('Longitude')
axs[0].set_ylabel('Latitude')

sns.scatterplot(data=df, x='longitude', y='latitude', hue='cluster_3',
                palette='viridis', ax=axs[1])
axs[1].set_title('Fit Data with K = 7')
axs[1].set_xlabel('Longitude')
axs[1].set_ylabel('Latitude')

plt.show()

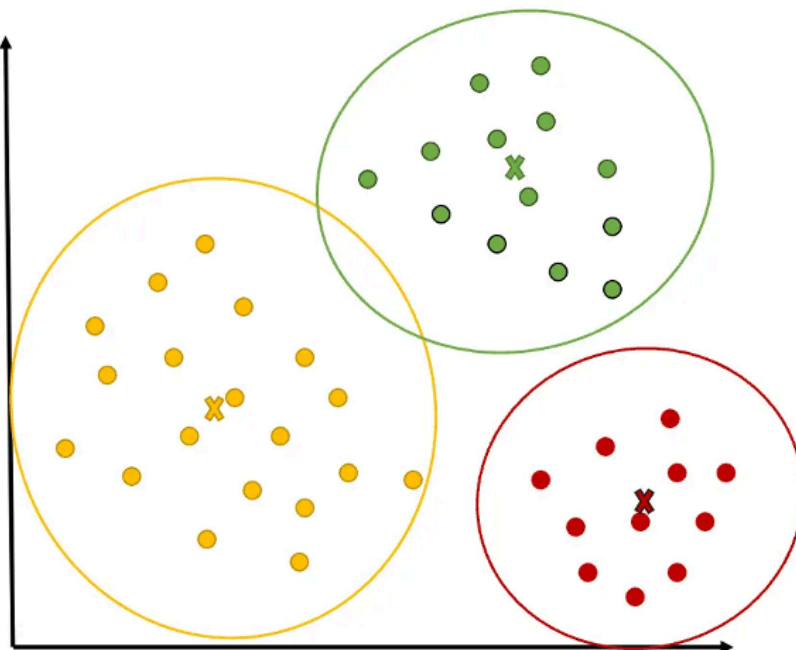
```



3. K-Means Clustering - The Elbow Method

At this point, you understand K-Means. You know how it groups data, how centroids move, how clusters are formed. But there is still one unresolved question:

How do we choose the “right” number of clusters (K)?



3 Clusters → But is it always enough?

Because let's be honest. If I give you a dataset and ask: "Should K be 2? 3? 7? 15?"

There's no immediate answer. And this is where the **Elbow Method** comes in.

The Elbow Method tries to answer one question:

■ **At what point does adding more clusters stop being useful?**

To measure this, we introduce something important: **Within-Cluster Sum of Squares (WCSS)** → also commonly referred to as **Inertia**

What is WCSS? WCSS measures how **compact** the clusters are. For each cluster:

- Take every data point
- Measure its distance to the centroid
- Square that distance
- Sum everything together

Mathematically, for K clusters:

$$WCSS = \sum_{i=1}^K \sum_{p_j \in C_i} d(p_j, C_i)^2$$

Let's slow this down: C_i → the i -th cluster; p_j → a point inside that cluster; $d(p_j, C_i)$ → distance from point to centroid

So WCSS is basically: "How far are points from their cluster centers?"

- **Low WCSS** → points are close to centroids → tight clusters
- **High WCSS** → points are spread out → loose clusters

Now here's something very important: As K **increases WCSS will always decrease**. Because:

- More clusters → smaller groups
- Smaller groups → points are closer to their centroids

In the extreme case:

- If K = number of data points → Each point is its own cluster → $WCSS = 0$

So minimizing WCSS alone is meaningless. We need a smarter way.

Instead of just looking at WCSS. We plot: $WCSS$ vs K . And something interesting happens.

- At first, increasing K causes a **sharp drop** in WCSS
- Then the improvement slows down
- Eventually, adding more clusters barely helps

This creates a curve that looks like an arm. And the key point is: **The “elbow”** is the point where the rate of decrease in WCSS **suddenly slows down**.

- Before the elbow → adding clusters is very useful
- After the elbow → diminishing returns

So we choose: K **at the elbow point**

How the Process Works: Now let's connect everything into a procedure.

- **Step 1 — Try Multiple Values of K**
 - We don't guess K once. We try many values: $K = 1, 2, 3, \dots, K_{max}$
- **Step 2 — Run K-Means for Each K** . For each value of K :
 - Fit the K-Means model
 - Compute its WCSS (inertia)
- **Step 3 — Store WCSS Values.** Now we have pairs like:
 - $(K = 1, WCSS = \dots)$
 - $(K = 2, WCSS = \dots)$
 - $(K = 3, WCSS = \dots)$
- **Step 4 — Plot the Curve.** Plot:
 - X-axis → K
 - Y-axis → $WCSS$
- **Step 5 — Find the Elbow.** Look for the point where:
 - The curve bends
 - The drop in $WCSS$ slows significantly→ That is your **optimal K** .

A Subtle but Important Detail is that: The Elbow Method depends entirely on WCSS. And WCSS depends on K-Means. Which means: You cannot use the Elbow Method *without running K-Means multiple times first*.

So the process is not: "**Use Elbow** → **get K** → **run K-Means**"

It is actually: "**Run K-Means many times** → **compute WCSS** → **then use Elbow to choose K**"

Elbow Method with Python - Now translating this idea into code:

- We define a function: `elbow_method(X)`. Inside this function:
 - Run K-Means for multiple values of K
 - Collect inertia (WCSS) for each run
 - Approximate the elbow point (often using discrete differences)
 - Plot the curve

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

def elbow_method(X):
    distortions = []
    # Perform K-Means on multiple k
    K = range(2, 16)
    for k in K:
        model = KMeans(n_clusters=k)
        model.fit(X)
        # Calculate their inertia (WCSS)
        distortions.append(model.inertia_)

    # Compute second derivative (discrete approximation)
    diff = np.diff(distortions, 2)
    elbow_point = np.argmax(diff) + 1 # offset due to diff

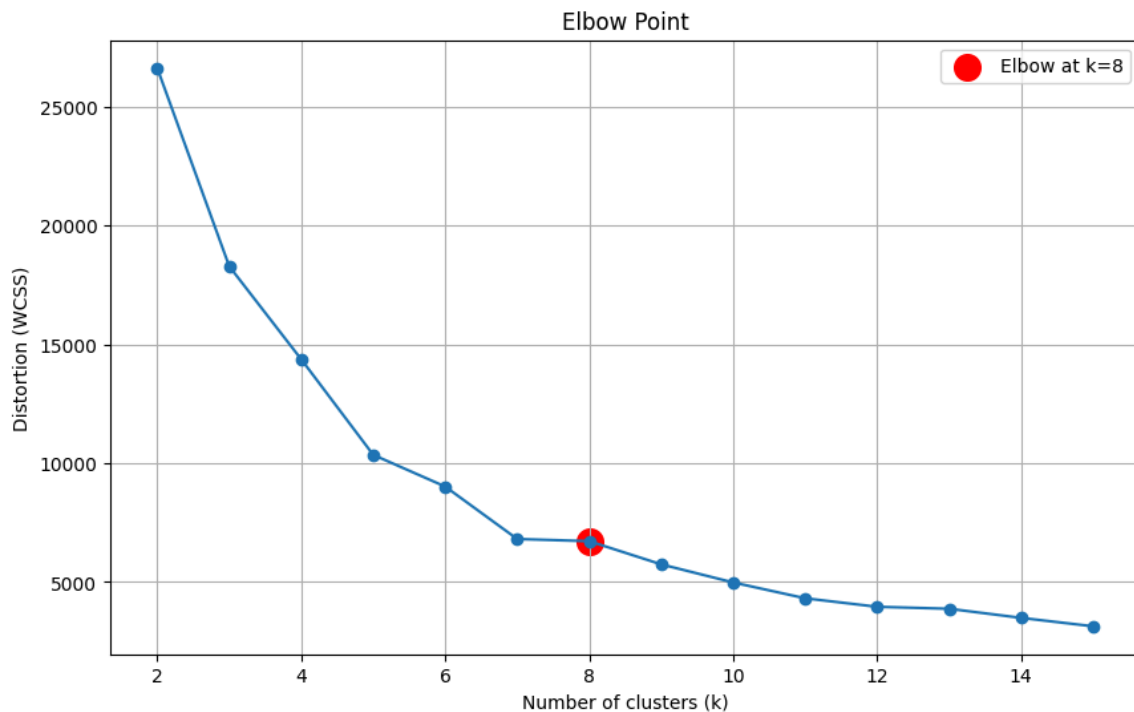
    # Plot
    plt.figure(figsize=(10, 6))
```

```

plt.plot(K, distortions, marker='o')
plt.scatter(K[elbow_point], distortions[elbow_point], col
or='red', s=200,
            label=f"Elbow at k={K[elbow_point]}")
plt.grid(True)
plt.title('Elbow Point')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Distortion (WCSS)')
plt.legend()
plt.show()

# Read data
df = pd.read_csv('housing.csv',
usecols=['longitude', 'latitude', 'median_house_value'])
# Cluster by geography
X = df[['longitude', 'latitude']]
scaler = StandardScaler() # Normalize the data
X_scaled = scaler.fit_transform(X)
elbow_method(X)

```



The elbow point can change every time you run the code again.
Why the **elbow point can change**? Because K-Means starts with **random centroid initialization**.

Different starting points → slightly different clusters → slightly different WCSS → the "elbow" might shift.

So don't treat the elbow as something absolute. It's a **guideline**

And that's it. The Elbow Method is not a strict rule. It doesn't "prove" the correct K . Instead, it gives you a **visual intuition**. Sometimes the elbow is clear. Sometimes, it's barely noticeable.

And in those cases? You are not just running an algorithm anymore. You are **interpreting the data**. Because in unsupervised learning. There is no ground truth. Only patterns. And your ability to recognize them.

4. K-Means Clustering - The Silhouette Method

Now, if the Elbow Method feels a bit *visual* and subjective. The **Silhouette Method** takes a more **quantitative approach**. Instead of just looking at cluster compactness. It evaluates **how well each individual data point fits into its cluster**.

For every data point, we ask: **"Am I closer to my own cluster... or to another cluster?"**

This is measured using the **Silhouette Score**, which ranges from:

- $+1 \rightarrow$ **Perfect clustering**
- $0 \rightarrow$ **On the boundary**
- $-1 \rightarrow$ **Probably in the wrong cluster**
 - *Silhouette* $\approx 1 \rightarrow$ The point is tightly grouped with its cluster $\rightarrow$ And far away from others
 - *Silhouette* $\approx 0 \rightarrow$ The point lies between clusters
 - *Silhouette* $\approx -1 \rightarrow$ The point is closer to another cluster than its own

For a data point i , the Silhouette score is:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

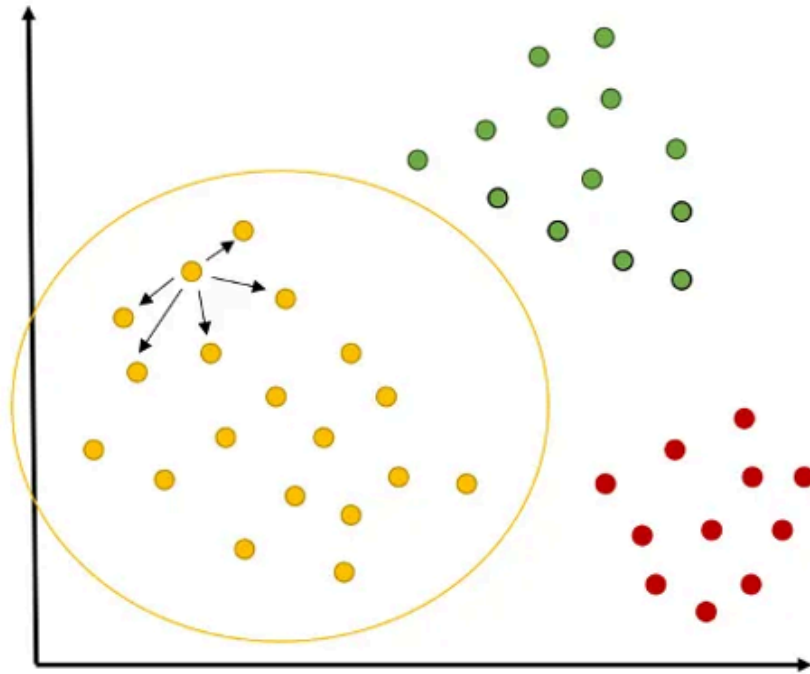
Where:

- $a(i) \rightarrow$ **Intra-cluster distance.** The average distance from point i to all other points in the same cluster
- $b(i) \rightarrow$ **Inter-cluster distance.** The average distance from point i to the *nearest neighboring cluster*

So in simple terms: $a(i) \rightarrow$ "How well do I fit with my own group?"; $b(i) \rightarrow$ "How close am I to the next best group?"

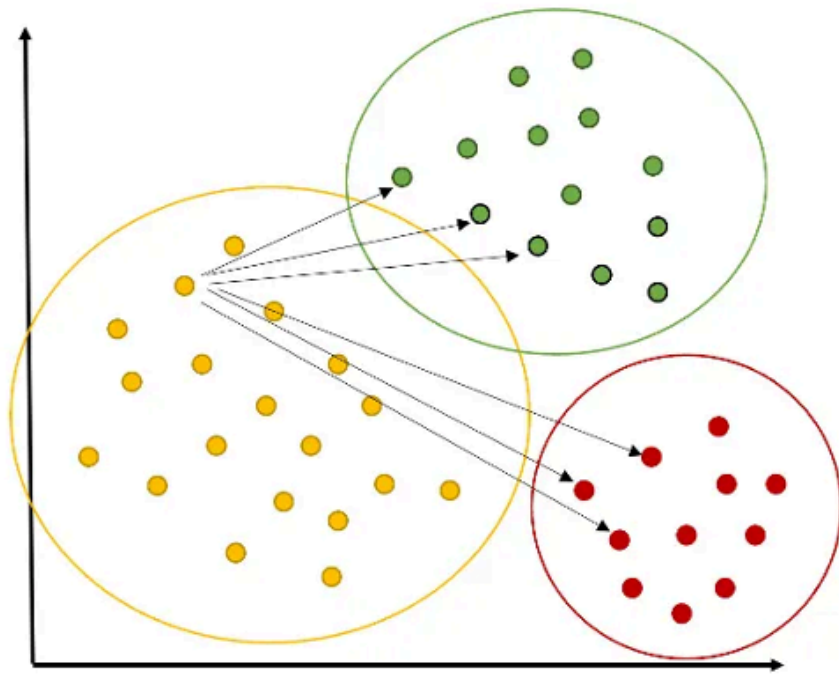
Let's walk through a single point.

- **Step 1 — Compute Intra-Cluster Distance** $a(i)$. Take your point.
 - Measure distance to all other points in the same cluster
 - Compute the average



→ This gives: "How tight is my cluster around me?"

- **Step 2 — Compute Inter-Cluster Distance $b(i)$.** Now compare against other clusters. For each *other* cluster:
 - Compute average distance to all its points
 - Then choose the **smallest** of those averages



→ This gives: ***“Which cluster is closest to me (besides my own)?”***

- **Step 3 — Compute the Score.** Plug into the formula. Now interpret:
 - If $b(i) > a(i)$ → score → close to 1
 - If $b(i) \approx a(i)$ → score → close to 0
 - If $b(i) < a(i)$ → score → **negative**

- **Step 4 — Compute for All Points. From: *Points* → *Clusters* → *Dataset*.** We don't stop at one point. We repeat the process for:
 - Every point in Cluster 1
 - Then Cluster 2
 - ... until all clusters are covered

- **Step 5 — Cluster Score.** For each cluster, take the **average Silhouette score** of its points

- **Step 6 — Overall Score.** Finally, we average all data point scores → gives a **single score for the dataset**
- **Choosing the Best K .** Now we repeat everything for different values of K .
 - $K = 2$ → compute score
 - $K = 3$ → compute score
 - $K = 4$ → compute score
 - ... and so on

And here's the key idea: **Choose the K with the highest Silhouette score.**

Because that means:

- Clusters are **tight** (low intra-distance)
- Clusters are **well-separated** (high inter-distance)

To visualize the Silhouette plot. We can plot K vs Silhouette score. Look for the **peak**. That peak corresponds to the best clustering structure.

Silhouette Method with Python - In practice, we define: `silhouette_method(X)`

Inside the function:

- Run K-Means for multiple values of K
- Compute Silhouette scores
- Plot them
- Highlight the best K (highest score)

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

def silhouette_method(X):
    silhouette_scores = []
    # Perform K-Means on multiple k
    K = range(3, 12)

    for k in K:
```

```

model = KMeans(n_clusters=k)
model.fit(X)

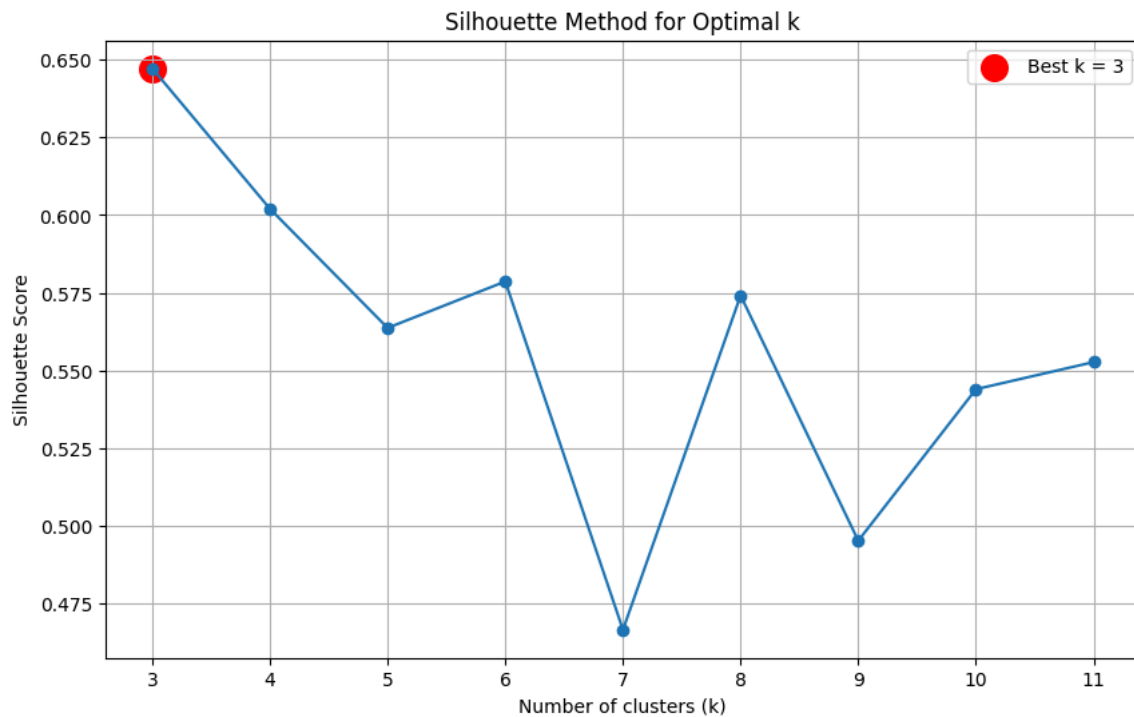
# Calculate and append silhouette score
score = silhouette_score(X, model.labels_, metric='euclidean')
silhouette_scores.append(score)

# Plot silhouette scores
plt.figure(figsize=(10, 6))
plt.plot(K, silhouette_scores, marker='o')
plt.title("Silhouette Method for Optimal k")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Silhouette Score")
plt.grid(True)

# Highlight best k
best_k = K[silhouette_scores.index(max(silhouette_scores))]
best_score = max(silhouette_scores)
plt.scatter(best_k, best_score, color='red', s=200, label=f"Best k = {best_k}")
plt.legend()
plt.show()

# Read data
df = pd.read_csv('housing.csv',
usecols=['longitude', 'latitude', 'median_house_value'])
# Cluster by geography
X = df[['longitude', 'latitude']]
scaler = StandardScaler() # Normalize the data
X_scaled = scaler.fit_transform(X)
silhouette_method(X)

```



Just like the Elbow Method, the **best K from the Silhouette Score can vary between runs** due to random centroid initialization in K-Means.

If the Elbow Method asks: "When do clusters stop improving significantly?". Then the Silhouette Method asks: ***"How good are the clusters, really?"***

One is about **structure**. The other is about **quality**. And when both methods agree? That's when you can trust your K .

But if they don't, then you're no longer just running algorithms. You're interpreting patterns. And that is where unsupervised learning becomes more art than science.

Limitations of Choosing K (Elbow & Silhouette)

Before we move on, there's something you should be aware of. Both the **Elbow Method** and the **Silhouette Method** are powerful. But they are not free.

- **Computational Cost:** Both methods require running K-Means **multiple times**. Not once. Not twice. But repeatedly. So instead of one clustering process
 - You now have many clustering processes + distance calculations

- And even if you limit the number of iterations. You optimize your code. It is still significantly more expensive than a single run of K-Means.
- **Not Ideal for Large Datasets:** Now imagine scaling this up. Thousands → manageable, but Millions → problematic
 - Because K-Means itself already computes distances for many points. Elbow & Silhouette repeat this process multiple times
 - Which leads to high computational cost, increased runtime, and heavy resource usage
 - So in large-scale scenarios... These methods can become impractical.

These methods don't just "find K ". They **search for it**. And searching always comes with a cost.

So while they are useful for understanding structure and exploring datasets. They are not always the best choice when speed matters and scale becomes too large

Summary

K-Means Clustering is our **starting point** into the world of unsupervised learning. It introduces a completely different mindset.

*No labels. No predictions. No ground truth. Instead, it focuses on one simple idea: **Grouping data based on similarity***

Through this section, you've seen:

- How K-Means works step by step
- How centroids move and clusters form
- How to implement it in practice
- And how to choose K using methods like Elbow and Silhouette

And despite its simplicity, K-Means is surprisingly powerful.

K-Means doesn't try to understand *what* the data means. It doesn't care about labels, categories, or names. It only asks: **"Which points belong together?"**

And sometimes, that's all you need to reveal structure hidden in data.

References

<https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/>

<https://www.geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k-in-kmeans/>

<https://www.ibm.com/think/topics/k-means-clustering>

<https://builtin.com/data-science/elbow-method>

<https://www.youtube.com/watch?v=4b5d3muPQmA>

<https://www.youtube.com/watch?v=GZj6ikx8PAc>

Next Section:

 [16. Density-Based Clustering](#)